

AI Workload Acceleration through Semantic Preserving Hardware Generation

Md Mahfuzul Haque • Md Rubel Ahmed
Louisiana Tech University

RAW 2026
PhD Forum

Research Problem & Motivation

- Demand for workload-aware accelerators is skyrocketing, but hand-crafting RTL requires extreme microarchitectural expertise
- LLMs can generate RTL from high-level descriptions, but their probabilistic token prediction lacks explicit execution models, leading to conservative, unoptimized pipelines.
- Conventional compilers strip high-level loop and dataflow structures during sequential lowering, weakening spatial mapping capabilities.

Central Research Question:

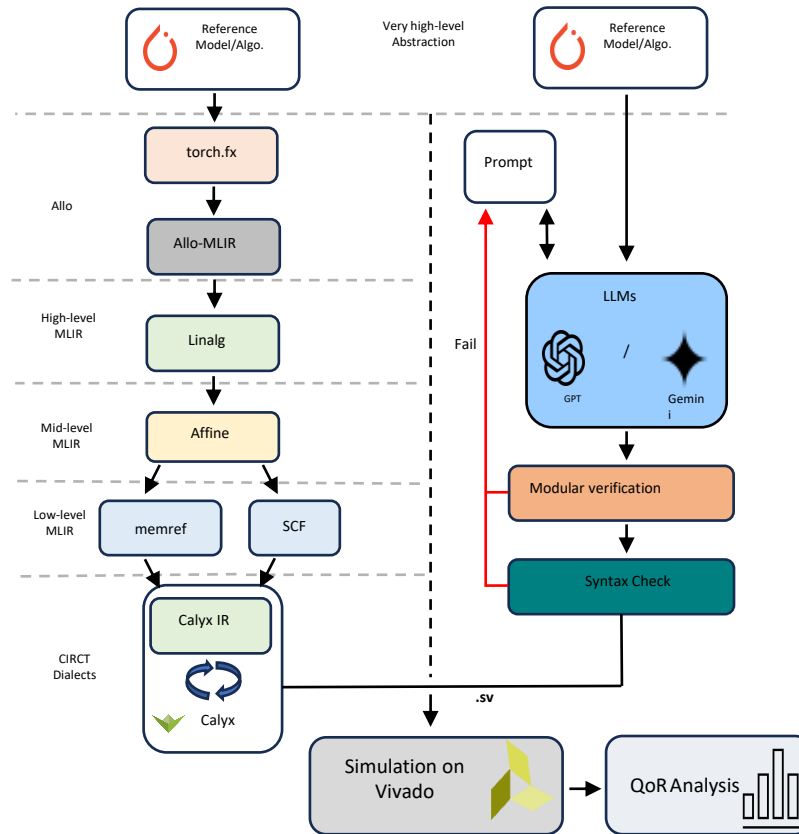
How can a hardware compiler preserve the high-level **semantic structure** of AI workloads while progressively lowering them into efficient, resource-aware spatial implementations for reconfigurable heterogeneous SoCs?

Key Challenges in Push-Button Design

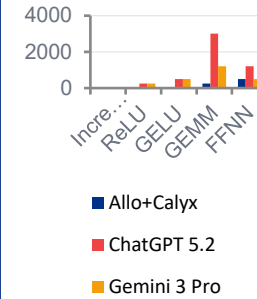
Resource utilization, clock frequencies, and critical timing paths are only accurately known after full downstream vendor synthesis.

Compiler must decide tiling, pipelining, unrolling, memory partitioning, and dataflow mapping without manual expert tuning.

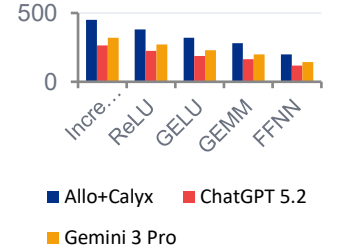
Methodology: Two RTL Generation Flows



LUT Efficiency



Performance



Key Findings

1.7×

higher Fmax
vs ChatGPT

75%

fewer LUTs
vs ChatGPT

1.4×

higher Fmax
vs Gemini

50%

fewer LUTs
vs Gemini

Compiler-driven flow consistently outperforms LLM-driven RTL generation across all benchmarks due to explicit scheduling, optimization passes, and semantic-preserving transformations.

Future Directions

Semantic-preserving compilation: Retain high-level dataflow across all lowering stages

Resource-aware DSE framework: Choose tiling, unrolling, pipelining, memory banking, and precision under FPGA constraints.

Push-Button HLS: Generate synthesizable hardware from high-level AI workload specifications.

LLMs as assistants: Use LLMs for front-end assistance, pragma suggestions, debugging, and design-space hints

For further details:

